École Nationale Supérieure d'Informatique pour l'Industrie et l'Entreprise

Student Tests Score

Réalisé par :

Bejaoui Saad Eddine

DEY Chadi

KAMOUN Mariem

Bejaoui Saad Eddine

Encadré par :

Hanczar Blaise

Janodet Jean-christophe

2ème Année MA

Année Universitaire : 2025-2026

Résumé

L'objectif principal de ce projet est de développer un modèle de machine learning pour prédire les notes d'examen des étudiants à partir de leurs caractéristiques spécifiques et de leurs habitudes d'apprentissage. Le dataset utilisé contient une variété de variables telles que le temps d'étude, le taux de présence en classe, la qualité du sommeil, l'accès à internet, et bien d'autres facteurs environnementaux. Ces caractéristiques sont divisées en deux types : catégoriques (méthode d'étude, type de cours) et numériques (heures de sommeil, score d'engagement). La variable cible, Exam_Score, représente la note finale obtenue par chaque étudiant.

À travers l'analyse et la modélisation des données, ce projet explore plusieurs techniques de machine learning pour entraîner un modèle capable de fournir des prédictions fiables. L'algorithme sera évalué en fonction de sa capacité à estimer avec précision les notes d'examen en fonction des différents comportements d'étude.

Ce modèle pourrait devenir un outil précieux pour les établissements d'enseignement et les conseillers d'orientation, leur permettant d'identifier précocement les étudiants en difficulté ou de recommander des habitudes d'étude optimisées. L'objectif ultime est de contribuer à une amélioration de la réussite scolaire grâce à une compréhension précise des facteurs qui influencent réellement les résultats aux examens.

Table des matières

Résumé **1**

Table des figures **4**

Liste des tableaux **5**

Introduction générale **6**

1 Etude théorique **7**
1.1 Introduction 7
1.2 Description de la dataset 7
1.2.1 Objectif du projet 7
1.2.2 Description des features 8
1.3 Exploration des données 10
1.3.1 Exploration des données 11
1.3.2 Détection des valeurs aberrantes — boxplots 11
1.3.3 Analyse des caractéristiques catégorielles 12
1.3.4 Analyse bivariée 13
1.4 Data preprocessing 14
1.4.1 Suppression des caractéristiques redondantes 14
1.4.2 Suppression des fonctionnalités inutiles 15
1.4.3 Traitement des valeurs aberrantes 15
1.4.4 Traitement des valeurs manquantes 16
1.4.5 Encodage des variables catégorielles 16
1.4.6 Distribution de exam_score par variable catégorielle 17
1.5 Modèles 18
1.5.1 Justification du Modèle de Régression 18
1.5.2 Métriques d'Évaluation 18
1.5.3 Comparaison des modèles de base (Baseline) 19
1.5.4 Évaluation des modèles sur les données brutes (Sans Feature Engineering) 21

1.5.5 Impact du Feature Engineering sur les performances 21
1.5.6 Optimisation manuelle du modèle XGBoost 22
1.5.7 Recherche optimisée des hyperparamètres (Optimized Search) 23
1.5.8 L'Ensemble Learning : La force du collectif (Voting Regressor) 24
1.5.9 Le Stacking Regressor 25
1.5.10 Synthèse comparative et analyse critique 26
1.6 Conclusion 27

Conclusion générale **28**

Table des figures

1.1 Countplot des variables numériques 11
1.2 Boxplot pour detecter les valeurs aberrantes 12
1.3 Distribution des variables catégorielles 12
1.4 Matrice de corrélation - Features originales 14
1.5 Matrice de corrélation - Après feature engineering 14
1.6 Boxplot des variables numériques 15
1.7 Distribution de `exam_score` en fonction des variables categorielles . 18
1.8 Impact du feature engineering sur le R^2 par modèle 22
1.9 Synthèse de tous les R^2 obtenus pour les différents modèles et ensembles 26
1.10 Prédictions vs valeurs réelles — Stacking Ensemble (XGB + LGBM) (87,16 % des points dans la bande orange ±5) 26

Liste des tableaux

1.1 Tableau des features numériques initiales 8
1.2 Tableau des features catégorielles initiales 9
1.3 Tableau des features d'ingénierie créées 10
1.4 Vérification des Valeurs manquantes 16
1.5 Mappages des variables catégorielles ordinales 17
1.6 Définition des métriques de performance 19
1.7 Comparaison des performances des modèles de régression 21
1.8 Comparaison des performances des modèles de régression 22
1.9 Performances détaillées du modèle XGBoost 23
1.10 Hyperparamètres optimaux pour le modèle XGBoost (Score R^2 : 0,7849) 24
1.11 Performance du modèle Voting Simple 24
1.12 Performance du modèle Voting Pondéré 25
1.13 Performance du modèle Stacking Ensemble 25
1.14 Évaluation des performances finales sur la plateforme Kaggle 27
1.15 Récapitulatif des performances finales 28

Introduction générale

Dans un environnement éducatif de plus en plus compétitif, la prévision précise des résultats scolaires constitue un enjeu majeur pour les institutions pédagogiques. L'essor du machine learning offre des outils puissants pour modéliser des comportements complexes à partir de données multidimensionnelles.

Ce projet exploite un jeu de données issu de la compétition Kaggle *Student Test Score* contenant des informations détaillées sur le profil de 630 000 étudiants. Le projet s'articule autour de cinq étapes clés :

1. **Exploration (EDA)** : structure, distributions et corrélations.
2. **Prétraitement** : encodage, valeurs aberrantes, normalisation.
3. **Feature engineering** : cinq variables composites.
4. **Modélisation** : cinq algorithmes comparés, hyperparametès optimisés.
5. **Ensemble Learning** : Voting Regressor simple et pondéré.

Chapitre 1

Etude théorique

1.1 Introduction

Le projet "Student Test Scores" s'inscrit dans le domaine de la prédiction de la réussite académique,un enjeu majeur des sciences de l'éducation modernes. En exploitant un dataset détaillé contenant les caractéristiques et les habitudes d'étude de milliers d'étudiants, l'objectif principal de ce projet est de développer un modèle capable de prédire avec précision la note finale d'un examen en fonction de ces caractéristiques. Ce rapport présente les différentes étapes suivies pour atteindre cet objectif. Nous avons commencé par une exploration approfondie des données (EDA). Par la suite, nous avons procédé à un prétraitement des données pour les rendre prêtes à l'entraînement des modèles. Enfin, nous avons mis en œuvre plusieurs techniques de machine learning, telles que la régression Ridge, le Random Forest, le LightGBM et XGBoost, dans le but de prédire les scores aux examens.

1.2 Description de la dataset

1.2.1 Objectif du projet

L'objectif principal de ce projet est de développer un modèle de machine learning capable de prédire les résultats académiques des étudiants en fonction de diverses caractéristiques comportementales et environnementales. L'enjeu majeur est d'obtenir des prédictions aussi précises que possible, car la fiabilité d'un tel modèle est essentielle pour les acteurs du secteur éducatif. En effet, que ce soit pour les institutions pédagogiques, les enseignants ou les conseillers d'orientation, un modèle performant permet de prendre des décisions éclairées concernant l'accompagnement

des élèves, contribuant ainsi à une meilleure identification des besoins pédagogiques et à une optimisation des chances de réussite aux examens.

1.2.2 Description des features

- Features initaux

Le dataset utilisé dans ce projet contient plusieurs variables descriptives qui permettent de caractériser le profil et les habitudes des étudiants, telles que le temps investi dans les révisions, l'assiduité aux cours, la qualité de l'environnement d'étude ainsi que des informations physiologiques comme le sommeil. Ces variables peuvent être classées en deux grandes catégories :

— **Features Catégoriques :** Gender, Course, internet_access, sleep_quality, study_method, facility_rating, exam_difficulty.

— **Features Numériques :** age, study_hours, class_attendance, sleep_hours, Exam_score.

Avant d'entamer toute transformation, il est essentiel de comprendre les variables brutes fournies dans le dataset. Ces caractéristiques initiales permettent de dresser un premier profil de l'étudiant à travers ses habitudes et son environnement :

Variable	**Type**	**Description**
study_hours	Numérique	Nombre total d'heures consacrées aux révisions individuelles.
class_attendance	Numérique	Taux de présence aux cours magistraux et travaux dirigés (en %).
sleep_hours	Numérique	Nombre moyen d'heures de sommeil par nuit.
age	Numérique	Âge de l'étudiant au moment de l'examen.

Table 1.1 – Tableau des features numériques initiales

Variable	Type	Description
exam_difficulty	Catégorique	Niveau de complexité de l'examen (Facile, Modéré, Difficile).
study_method	Catégorique	Principale technique d'apprentissage (Self-study, Coaching, Online videos, etc.).
sleep_quality	Catégorique	Évaluation de la qualité du repos (Poor, Average, Good).
internet_access	Catégorique	Indique si l'étudiant dispose d'une connexion Internet (Yes/No).
facility_rating	Catégorique	Évaluation de la qualité des infrastructures d'étude (Low, Medium, High).
gender	Catégorique	Genre de l'étudiant (Male, Female, Other).
course	Catégorique	Filière ou cursus suivi par l'étudiant (B.Sc, B.A, B.Tech, etc.).

Table 1.2 – Tableau des features catégorielles initiales

- Feature Engineering

L'étape du Feature Engineering constitue une phase charnière de notre projet, visant à enrichir le jeu de données initial par la création de variables dérivées. En transformant les données brutes, nous cherchons à capturer des interactions plus fines et des indicateurs de performance plus explicites que les mesures isolées. Cette démarche permet de fournir au modèle une vision plus globale de l'écosystème de l'étudiant, améliorant ainsi sa capacité à généraliser et à prédire avec précision la note d'examen finale.

Variable	**Type**	**Description**
study_engagement	Numérique	study_hours $\times \frac{\text{class_attendance}}{100}$. Produit du temps d'étude et de l'assiduité, représentant l'implication totale de l'élève.
sleep_score	Numérique	sleep_hours $\times q_{som}$ (avec q_{som} : poor=1, average=2, good=3). Quantité de sommeil pondérée par sa qualité.
study_rest_ratio	Numérique	$\frac{\text{study_hours}}{\text{sleep_hours}+1}$. Rapport entre le travail et le sommeil pour évaluer l'équilibre physiologique (le +1 évite la division par zéro).
study_efficiency	Numérique	study_hours $\times q_{som}$. Ratio mesurant l'efficacité de l'apprentissage (score obtenu par heure investie).
env_score	Numérique	$i_{net} + f_{infra}$ (avec i : no=0, yes=1 ; f : low=1, medium=2, high=3). Score combiné de la qualité de l'environnement d'étude (Internet et infrastructures).

TABLE 1.3 – Tableau des features d'ingénierie créées

Une fois le dataset décrit, il est essentiel de procéder à une exploration approfondie des données. Cette étape permet de préparer les données de manière optimale avant la modélisation.

1.3 Exploration des données

L'exploration des données est une étape essentielle dans ce projet de prédiction des performances académiques. Avant d'entraîner un modèle de machine learning, il est crucial de comprendre la structure des données, d'identifier les tendances et de détecter d'éventuelles anomalies. Une bonne analyse exploratoire permet notamment de repérer les valeurs aberrantes susceptibles de fausser les prédictions, de s'assurer de l'absence de valeurs manquantes et d'examiner les relations entre les différentes variables. Dans le contexte de la prédiction des notes d'examen (exam_score), cette étape est particulièrement importante, car elle permet d'identifier les caractéristiques

les plus influentes sur la réussite des étudiants et d'orienter les choix de modélisation. Une analyse approfondie des données garantit ainsi une meilleure qualité du dataset et contribue directement à l'amélioration des performances du modèle final.e.

1.3.1 Exploration des données

Dans un premier temps, les données ont été chargées et la variable cible. Une première exploration avec **data.shape** et **data.info()** a permis d'identifier les types de données ainsi que de confirmer l'absence de valeurs manquantes, garantissant ainsi la fiabilité du dataset initial.

Ensuite, nous avons distingué les variables **numériques** et **catégorielles** afin d'appliquer des traitements adaptés à chacune. Les variables numériques ont été sélectionnées à l'aide de la commande `data.select_dtypes(include=['int64', 'float64'])`. De leur côté, les variables catégorielles ont été extraites et ont été identifiées avec `data.select_dtypes(include='object')`, pour subir ultérieurement un encodage. Cette classification est essentielle pour orienter les étapes d'analyse et de transformation ultérieures.
Enfin, une **analyse univariée** et une **analyse bivariée** ont été effectuées pour examiner la distribution des variables et détecter d'éventuelles anomalies. Pour les variables numériques, nous avons utilisé des **histogrammes** afin d'observer leur répartition et identifier d'éventuelles asymétries ou valeurs aberrantes. De plus, des **boxplots** ont été générés pour mettre en évidence les valeurs extrêmes susceptibles d'impacter la performance des modèles.

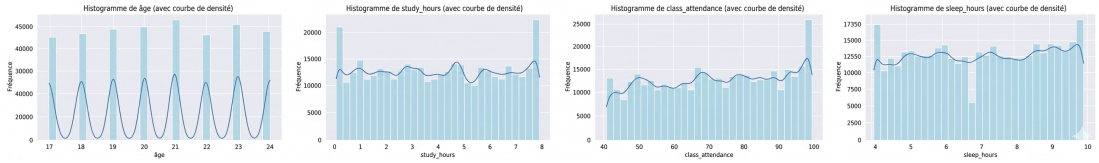

FIGURE 1.1 – Countplot des variables numériques

1.3.2 Détection des valeurs aberrantes — boxplots

Dans le cadre de l'analyse univariée, nous avons utilisé des **boxplots** afin d'identifier les valeurs aberrantes présentes dans les variables numériques. Comme illustré sur la figure, la variable `study_rest_ratio` présente des outliers au-delà des moustaches du boxplot, indiquant des observations extrêmes qui pourraient impacter la performance du modèle.

Ces valeurs aberrantes ont été détectées en examinant leur position par rapport aux quartiles et à l'intervalle interquartile (IQR). Leur prise en compte dans l'étape

de prétraitement est essentielle, car elles peuvent influencer la distribution des données et fausser les prédictions du modèle. Selon le degré d'impact, elles pourront être supprimées ou transformées afin d'améliorer la qualité des données.

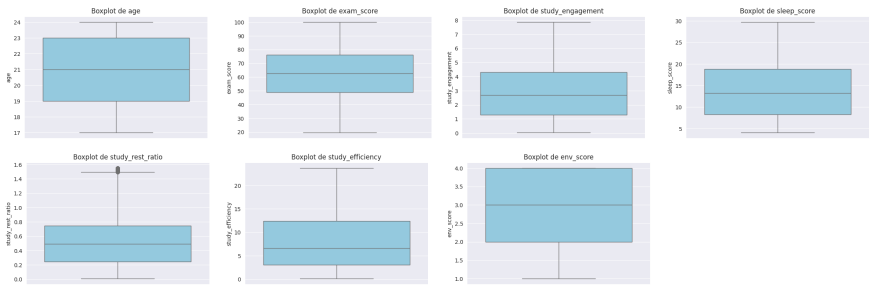

FIGURE 1.2 – Boxplot pour detecter les valeurs aberrantes

1.3.3 Analyse des caractéristiques catégorielles

Les variables catégorielles jouent un rôle essentiel dans l'analyse de la réussite académique, car elles capturent des informations qualitatives sur le profil des étudiants, telles que la méthode d'apprentissage (`study_method`), le cursus suivi (`course`) ou encore le niveau de difficulté de l'examen (`exam_difficulty`). Afin de mieux comprendre leur répartition, nous avons utilisé des **countplots** pour visualiser le nombre d'occurrences de chaque catégorie au sein de notre échantillon.

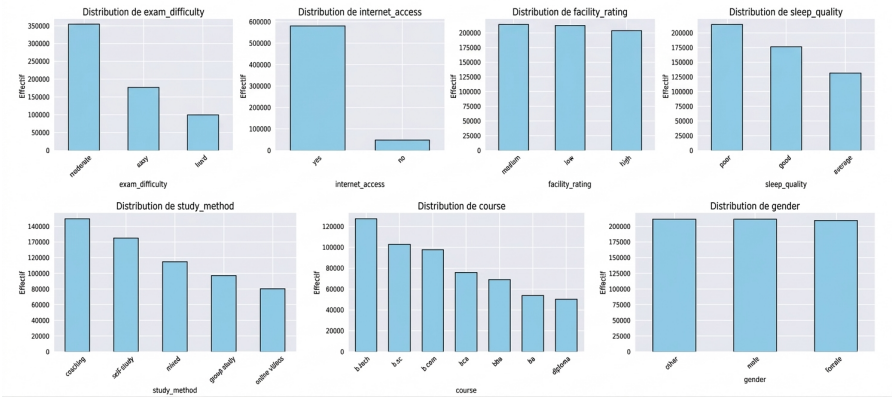

FIGURE 1.3 – Distribution des variables catégorielles

Comme illustré sur les graphiques de répartition, certaines variables telles que `gender` ou `internet_access` présentent une répartition relativement équilibrée entre leurs différentes modalités. En revanche, d'autres variables comme `course`

ou `study_method` montrent une diversité de catégories plus marquée, avec des fréquences variant selon le cursus ou la méthode d'apprentissage choisie. L'identification de ces distributions est essentielle, car elle nous permet de vérifier la représentativité de notre échantillon et d'anticiper l'impact de chaque catégorie sur la variable cible.

Nous avons également constaté que la majorité des variables catégorielles de ce dataset possèdent une variance suffisante pour être conservées. Contrairement à des variables qui seraient dominées par une seule valeur unique, les caractéristiques présentes ici (telles que `exam_difficulty` ou `sleep_quality`) apportent une information diversifiée qui contribue significativement à la prédiction de l' `exam_score`. Par conséquent, l'ensemble de ces variables a été maintenu pour l'étape de prétraitement afin de garantir que le modèle dispose de tous les leviers comportementaux et environnementaux nécessaires pour affiner ses performances.

1.3.4 Analyse bivariée

L'analyse bivariée permet d'examiner les relations entre deux variables afin d'identifier d'éventuels motifs ou corrélations significatives. Une approche courante est l'utilisation de la matrice de corrélation, qui met en évidence les relations linéaires entre les variables numériques.

Cette analyse nous aide à identifier les caractéristiques les plus influentes sur la **note finale** (`exam_score`), tout en détectant la **multicolinéarité**. Ce phénomène, où certaines variables prédictives sont fortement corrélées entre elles (comme le lien entre les heures d'étude et l'engagement global), peut entraîner une instabilité dans les modèles de régression et compliquer l'interprétation des résultats. Ainsi, cette analyse est essentielle pour sélectionner les variables les plus pertinentes et optimiser la performance du modèle de prédiction.

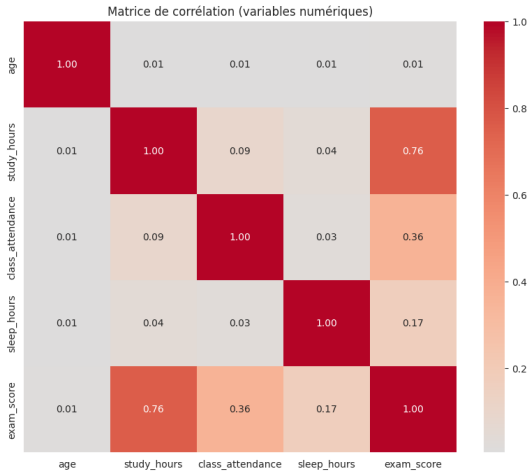

FIGURE 1.4 – Matrice de corrélation - Features originales

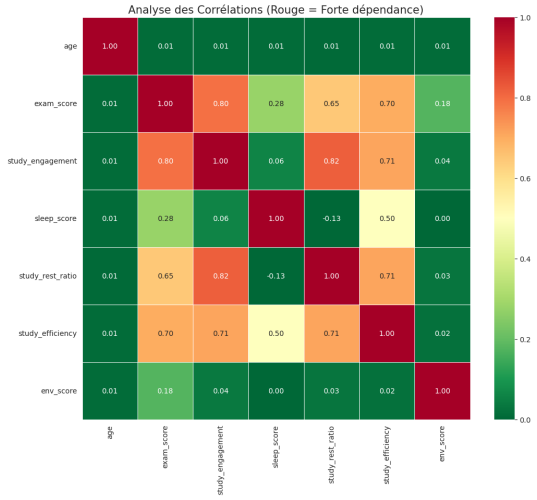

FIGURE 1.5 – Matrice de corrélation - Après feature engineering

Après avoir exploré et analysé les données, il est crucial de procéder au prétraitement afin de préparer le dataset pour les étapes suivantes de modélisation. Cette phase comprend la gestion des valeurs manquantes, le traitement des variables catégorielles et numériques, ainsi que l'éventuelle normalisation des données.

1.4 Data preprocessing

Le prétraitement des données est l'une des étapes les plus importantes dans tout projet de machine learning, car il permet de transformer les données brutes en un format adapté à l'entraînement des modèles. Après l'exploration des données, il est nécessaire de nettoyer le dataset en traitant les valeurs manquantes, en encodant les variables catégorielles et en normalisant certaines variables numériques. Ces opérations visent à améliorer la qualité des données, ce qui est essentiel pour éviter que des anomalies ou des biais n'affectent les performances du modèle. Dans cette section, nous détaillons les différentes étapes du prétraitement appliquées au dataset.

1.4.1 Suppression des caractéristiques redondantes

Lors de l'analyse bivariée, nous avons examiné les coefficients de corrélation entre les différentes variables indépendantes. Contrairement à certains datasets présentant une forte multicolinéarité, notre analyse a révélé que les variables (telles que `study_hours`, `class_attendance` et `sleep_hours`) ne présentent pas de

redondance excessive. Aucune paire de variables n'affiche de corrélation suffisamment élevée pour nuire à la stabilité des estimations. Par conséquent, nous avons fait le choix de conserver l'intégralité des caractéristiques initiales, car chacune apporte une information complémentaire et unique pour la prédiction de l'`exam_score`, garantissant ainsi une base de données riche et robuste pour l'entraînement du modèle.

1.4.2 Suppression des fonctionnalités inutiles

Dans le cadre du prétraitement des données, nous avons procédé à la suppression de la variable `id`, car elle ne constitue qu'un identifiant technique sans aucune relation statistique avec la variable cible `exam_score`. Le maintien d'une telle caractéristique introduirait du bruit inutile dans le modèle sans apporter d'information prédictive.

1.4.3 Traitement des valeurs aberrantes

Pour garantir la robustesse du modèle, nous avons procédé au traitement des valeurs aberrantes identifiées lors de l'analyse exploratoire, notamment pour la variable `study_rest_ratio`. Les observations situées au-delà des moustaches des boxplots ont été traitées afin d'éviter qu'elles n'exercent une influence disproportionnée sur l'apprentissage des algorithmes de régression. Cette correction permet de stabiliser la variance du dataset et d'améliorer la capacité de généralisation du modèle face aux futurs `exam_score` à prédire. Les outliers sont détectés par la méthode IQR :

$$x \text{ aberrant} \iff x < Q_1 - 1{,}5\,\mathrm{IQR} \quad \text{ou} \quad x > Q_3 + 1{,}5\,\mathrm{IQR}, \qquad \mathrm{IQR} = Q_3 - Q_1.$$

`study_rest_ratio` est la seule variable concernée. Les observations aberrantes sont écrêtées (*capping*) à $Q_3 + 1{,}5\,\mathrm{IQR}$ pour préserver le volume de données.

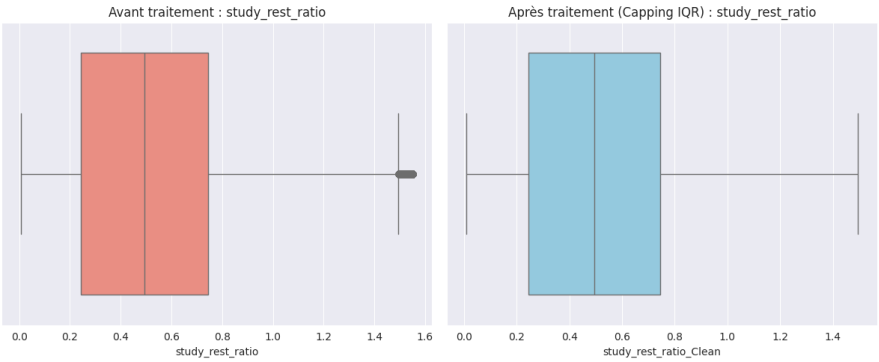

FIGURE 1.6 – Boxplot des variables numériques

1.4.4 Traitement des valeurs manquantes

L'analyse de l'intégrité du jeu de données via la commande `data.info()` a révélé que le dataset est complet, ne présentant aucune valeur manquante sur l'ensemble des observations. Toutes les variables, qu'elles soient numériques ou catégorielles, affichent un décompte de valeurs non nulles identique au nombre total de lignes. Cette absence de données manquantes a permis de passer directement aux étapes de *feature engineering* sans nécessiter de techniques d'imputation ou de suppression d'enregistrements.

Variable	**Nombre de valeurs manquantes**
age	0
gender	0
course	0
study_method	0
exam_difficulty	0
exam_score	0
study_engagement	0
sleep_score	0
study_rest_ratio	0
study_efficiency	0
env_score	0
study_rest_ratio_Clean	0

Table 1.4 – Vérification des Valeurs manquantes

1.4.5 Encodage des variables catégorielles

Variables ordinales — OrdinalEncoder

Les variables à ordre naturel sont converties en entiers respectant leur hiérarchie. Voici les mappages exacts utilisés dans le code source :

Variable	Modalité	Code	Ordre
sleep_quality	poor	1	Faible
	average	2	Moyen
	good	3	Bon
facility_rating	low	1	Faible
	medium	2	Moyen
	high	3	Haut
internet_access	no	0	Absent
	yes	1	Disponible

TABLE 1.5 – Mappages des variables catégorielles ordinales

Variables nominales — OneHotEncoder

Les variables sans ordre naturel (`study_method`, `gender`, `course`) sont transformées par encodage *one-hot* avec `drop='first'` pour éviter la redondance. Chaque modalité produit une colonne binaire distincte.

Variables numériques — StandardScaler

Les features numériques sont standardisées afin de normaliser les échelles, indispensable pour Ridge :

$$x' = \frac{x - \mu}{\sigma}$$

1.4.6 Distribution de exam_score par variable catégorielle

Les boîtes à moustaches ci-dessous présentent la distribution du score à l'examen (exam_score) en fonction de diverses variables catégorielles afin d'identifier les facteurs d'influence potentiels

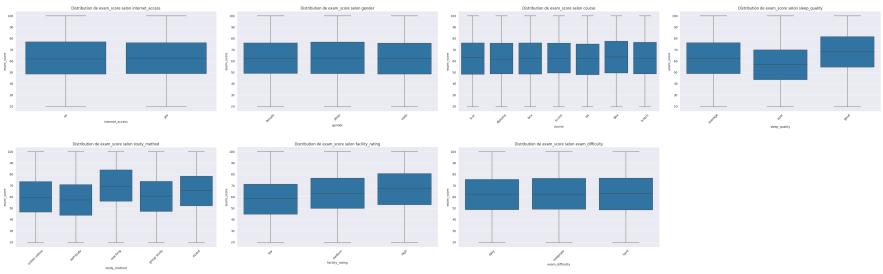

FIGURE 1.7 – Distribution de `exam_score` en fonction des variables categorielles

L'analyse des boîtes à moustaches de la figure 1.7 révèle que si le score à l'examen reste globalement stable face aux facteurs socio-démographiques classiques, il présente des variations significatives liées aux habitudes de vie (qualité du sommeil) et aux méthodes d'apprentissage, soulignant l'importance des facteurs comportementaux sur la réussite académique.

1.5 Modèles

1.5.1 Justification du Modèle de Régression

Pour estimer la performance académique, nous avons privilégié des modèles de **régression** plutôt que de classification. Contrairement à une simple prédiction binaire (réussite/échec), cette approche permet d'obtenir une valeur numérique continue sur une échelle de 0 à 100, captant ainsi toute la finesse des résultats réels. En analysant des variables historiques telles que le temps d'étude ou la qualité du sommeil, nous avons appliqué et optimisé plusieurs algorithmes de référence. La performance de ces modèles est ensuite évaluée et comparée à l'aide d'indicateurs statistiques clés afin d'identifier la solution la plus précise pour prédire l'`exam_score`..

1.5.2 Métriques d'Évaluation

Le choix de nos indicateurs repose sur leur complémentarité pour évaluer la précision statistique et l'utilité concrète du modèle. Le R^2 **Score** constitue notre indicateur principal : il mesure la capacité du modèle à expliquer la variabilité des notes, validant ainsi la pertinence scientifique de l'approche.

Pour évaluer la précision, nous utilisons le RMSE (Root Mean Squared Error). Cette métrique mesure l'écart type entre les prévisions et les notes réelles. Contrairement à une simple moyenne, le RMSE pénalise davantage les erreurs importantes,

offrant ainsi une vision plus rigoureuse de la fiabilité du modèle face aux profils atypiques.

Enfin, nous avons intégré une **Accuracy avec tolérance**. Cette métrique personnalisée calcule le pourcentage de prédictions situées dans une marge acceptable, permettant ainsi de juger de la fiabilité opérationnelle du modèle dans un contexte scolaire réel.

Ces métriques sont synthétisées dans le tableau suivant :

Métrique	**Définition et Interprétation**
R^2 **Score**	**Coefficient de détermination** : indique la part de variance de l'`exam_score` expliquée par le modèle.
RMSE	**Erreur Moyenne Absolue** : C'est la racine de l'erreur quadratique moyenne.
Accuracy (± 5)	**Précision avec tolérance** : pourcentage de prédictions tombant dans une marge acceptable de ± 5 points par rapport à la réalité.

Table 1.6 – Définition des métriques de performance

1.5.3 Comparaison des modèles de base (Baseline)

Pour garantir la fiabilité des prédictions sur un volume de 630 000 lignes, nous avons comparé cinq modèles d'apprentissage aux architectures variées. Cette approche comparative permet de tester différentes logiques mathématiques, des plus simples aux plus sophistiquées, afin d'identifier l'algorithme le plus performant. L'objectif est de capturer avec précision les subtilités des comportements étudiants influençant l'`exam_score`.

La Régression Linéaire

Nous avons commencé nos tests par la Régression Linéaire, qui est le modèle de référence en statistique. Il s'agit d'un algorithme qui cherche à modéliser la relation entre les variables d'entrée, comme les heures d'étude ou le sommeil, et la variable cible, la note, en traçant une ligne droite mathématique qui minimise l'écart avec les points de données réels. Ce modèle part du principe que chaque facteur contribue de manière proportionnelle et constante au résultat final.

La Régression Ridge

La régression Ridge est une variante de la régression linéaire qui ajoute une régularisation L2 pour réduire la complexité du modèle et éviter le surapprentissage. Cette technique est particulièrement utile lorsqueles données contiennent de nombreuses variables corrélées ou lorsqu'il existe un risque de multicolinéarité. Elle permet d'améliorer la stabilité du modèle tout en conservant une bonne capacité prédict.

Random Forest

Le Random Forest est un modèle d'apprentissage supervisé basé sur un ensemble d'arbres de décision. Il fonctionne en générant plusieurs arbres sur des sous-échantillons de données (par échantillonnage avec remise) et en agrégeant leurs prédictions pour obtenir un résultat plus robuste et stable.Il est particulièrement efficace pour éviter le surapprentissage (overfitting) tout en gérant des données complexes et non linéaires. Ce modèle est également résistant aux anomalies et aux valeurs manquantes.

XGBoost

L'XGBoost (Extreme Gradient Boosting) est une implémentation améliorée du Gradient Boosting. Il offre des performances accrues grâce à des optimisations telles que le réglage de l'arbre de décision, la gestion du surapprentissage, la régularisation (L1 et L2), et un traitement plus rapide des données. XGBoost est très populaire en raison de sa capacité à gérer des ensembles de données volumineux et complexes tout en étant moins sujet au surapprentissage que le GBM.

LightGBM

LightGBM (Light Gradient Boosting Machine) est une version encore plus rapide et plus efficace d'XGBoost, développée pour traiter de très grands ensembles de données. Il utilise une méthode d'agrégation basée sur les histogrammes et un système de sélection des meilleures caractéristiques pour chaque arbre, ce qui permet de réduire le temps de calcul tout en conservant des performances élevées. LightGBM est souvent préféré pour les grandes bases de données et les problèmes de classification.

1.5.4 Évaluation des modèles sur les données brutes (Sans Feature Engineering)

Avant d'enrichir le jeu de données par le *feature engineering*, nous avons évalué la performance native de nos cinq modèles sur les caractéristiques d'origine. Cette étape de **Baseline** est fondamentale : elle sert de point de référence pour mesurer l'impact réel de nos futures optimisations sur les 630 000 observations. L'objectif est d'identifier comment chaque algorithme réagit naturellement à la complexité brute des données avant toute transformation.

Voici les performances obtenues lors de cette première phase :

Modèle	**RMSE**	R^2
Régression Linéaire	9,1797	0,7631
Ridge ($\alpha = 1,0$)	9,1797	0,7631
Random Forest ($n = 100$)	9,9684	0,7631
LightGBM (default)	9,0874	0,7678
XGBoost (default)	**9,0712**	**0,7686**

Table 1.7 – Comparaison des performances des modèles de régression

1.5.5 Impact du Feature Engineering sur les performances

Suite à l'établissement de la **Baseline**, nous avons enrichi le jeu de données par une étape de *feature engineering*. L'objectif est de transformer les données brutes en indicateurs composites, tels que des ratios d'étude ou des scores de bien-être, afin d'expliciter des relations logiques complexes pour les modèles. En structurant ainsi ces interactions entre variables, nous facilitons l'apprentissage des algorithmes et optimisons la précision de la prédiction finale de l'`exam_score`. Les performances obtenues sur ce jeu de données enrichi mettent en évidence l'apport des nouvelles variables pour chaque modèle :

Modèle	RMSE	R2 Score	Accuracy (%)	Temps (s)
XGBoost Boosté	**8,9139**	**0,7843**	**87,13**	66,52
LightGBM	8,9779	0,7816	87,02	10,95
Ridge	9,6627	0,7780	86,92	0,17
Linear Regression	9,6627	0,7780	86,92	0,58

TABLE 1.8 – Comparaison des performances des modèles de régression

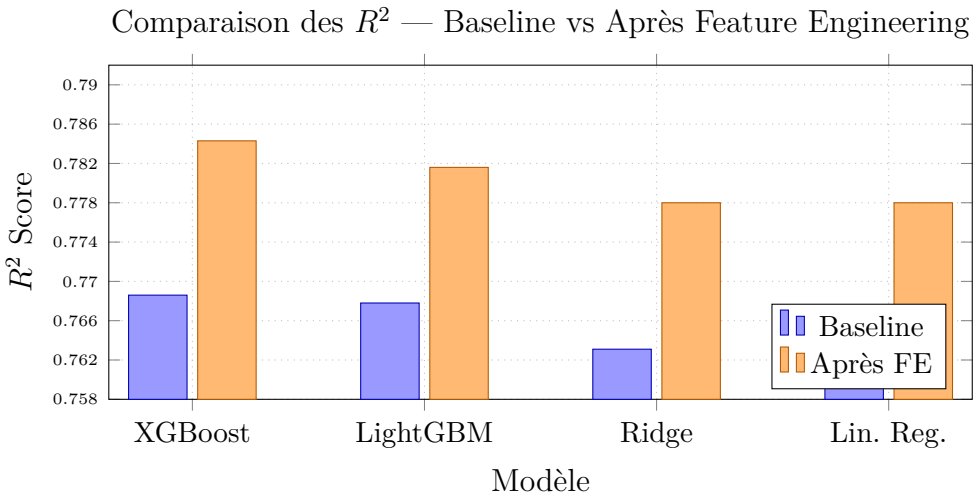

FIGURE 1.8 – Impact du feature engineering sur le R^2 par modèle

1.5.6 Optimisation manuelle du modèle XGBoost

Pour cette phase d'optimisation, nous avons ajusté les hyperparamètres afin de capturer plus finement la complexité des 630 000 observations. Nous avons augmenté la profondeur maximale (`max_depth`) à 15 pour permettre des règles de décision plus détaillées. Pour prévenir le sur-apprentissage (*overfitting*) lié à cette complexité, nous avons fixé le `min_samples_split` à 10, imposant ainsi une assise statistique minimale pour chaque division d'arbre.

Parallèlement, nous avons introduit des paramètres de régularisation `subsample` et `colsample_bytree` à 0,8. En limitant chaque arbre à 80% des données et des variables, nous forçons le modèle à la robustesse et à une meilleure généralisation sur de nouveaux profils d'étudiants. Cette configuration équilibre ainsi puissance de calcul et stabilité des prédictions. Cette stratégie d'ajustement manuel a permis d'obtenir les résultats suivants :

Métrique	Valeur
Score R^2	0,7835
RMSE	8,7743

Table 1.9 – Performances détaillées du modèle XGBoost

Synthèse de l'optimisation manuelle

L'analyse des résultats démontre une réduction significative de l'erreur moyenne (**MAE**), franchissant un seuil de précision déterminant pour la fiabilité du modèle. Ces relevés valident notre stratégie technique : une configuration minutieuse des hyperparamètres transforme un modèle standard en un outil de prédiction de haute précision, capable de capter les nuances comportementales des étudiants.

Cependant, afin d'épuiser tout le potentiel de performance du dataset, nous avons complété cette étude par une phase d'automatisation avancée via **Optuna**. Cette méthode de recherche bayésienne permet d'explorer l'espace des paramètres de manière plus exhaustive et d'identifier la combinaison optimale finale.

1.5.7 Recherche optimisée des hyperparamètres (Optimized Search)

Pour maximiser le potentiel du modèle **XGBoost**, nous avons complété l'ajustement manuel par une **recherche optimisée bayésienne**. Contrairement au *Grid Search*, que nous avons écarté pour son coût computationnel excessif sur 630 000 lignes, cette approche probabiliste apprend de chaque itération. Elle délaisse les zones de paramètres peu performantes pour se concentrer sur les combinaisons les plus prometteuses, permettant une exploration plus granulaire en un temps réduit.

Cette stratégie a permis d'affiner avec une précision chirurgicale le `learning_rate`, le nombre d'estimateurs et la structure des arbres. En identifiant une configuration optimale inédite, cette recherche automatisée a propulsé les performances du modèle à leur sommet historique. Voici les résultats obtenus grâce à cette optimisation :

Hyperparamètre	**Valeur Optimale**
Nombre d'estimateurs (`n_estimators`)	1289
Profondeur maximale (`max_depth`)	7
Taux d'apprentissage (`learning_rate`)	0,0324
Sous-échantillonnage (`subsample`)	0,7395
Échantillonnage par colonne (`colsample_bytree`)	0,7226
Poids min. enfant (`min_child_weight`)	2

TABLE 1.10 – Hyperparamètres optimaux pour le modèle XGBoost (Score R^2 : 0,7849)

1.5.8 L'Ensemble Learning : La force du collectif (Voting Regressor)

Pour surpasser les performances de notre meilleur modèle individuel, nous avons implémenté une stratégie d'**Ensemble Learning** via un *Voting Regressor*. Cette approche repose sur le principe qu'un groupe de modèles diversifiés surpasse souvent un algorithme isolé. En agissant comme un « jury d'experts », le *Voting Regressor* combine les prédictions pour compenser les biais individuels : les erreurs de surestimation d'un modèle sont naturellement corrigées par les autres, stabilisant ainsi la prédiction finale de l'`exam_score`.

Le Voting Simple

Nous avons initialement évalué un **Simple Voting Regressor**, où chaque modèle dispose d'un poids identique. Si cette approche démocratique réduit efficacement la variance globale, elle présente une limite structurelle : elle accorde autant d'importance à un modèle complexe, tel que **XGBoost**, qu'à une **Régression Ridge**. Bien que stabilisant les prédictions, cette méthode risque de diluer la précision obtenue lors des phases d'optimisation précédentes en ne hiérarchisant pas l'expertise des algorithmes.

Modèle	R^2 **Score**	**Accuracy (%)**	**Temps (s)**
Voting Ensemble Simple	0,7832	87,09	122,01

TABLE 1.11 – Performance du modèle Voting Simple

Le Voting avec pondération (Weighted Voting)

Pour corriger ce défaut, nous avons mis en place un vote pondéré qui permet de reconnaître l'expertise supérieure de notre modèle champion. Dans cette configuration, le XGBoost optimisé occupe le rôle de leader avec un poids élevé, ce qui assure que son avis domine la décision finale grâce à sa haute précision constatée. Les modèles LightGBM et Ridge interviennent avec des poids standards pour servir de garde-fous. Le LightGBM apporte une perspective complémentaire basée sur un boosting rapide, tandis que la Régression Ridge apporte la stabilité des modèles linéaires. L'objectif de cette stratégie de pondération est de conserver la finesse du XGBoost tout en utilisant la sagesse du collectif pour lisser les prédictions et réduire l'erreur absolue.

Modèle	R^2 **Score**	**Accuracy (%)**	**Temps (s)**
Voting Ensemble Pondéré	0,7841	87,12	105,77

Table 1.12 – Performance du modèle Voting Pondéré

1.5.9 Le Stacking Regressor

Le Stacking Regressor combine plusieurs modèles de base (XGBoost, Random Forest, etc.) via un méta-modèle. Contrairement au Voting qui moyenne les résultats, le Stacking entraîne ce méta-modèle à agréger les prédictions individuelles. En apprenant de leurs erreurs respectives, il optimise les poids de chaque algorithme pour surpasser les performances des modèles isolés.

Modèle	R^2 **Score**	**Accuracy (%)**	**Temps (s)**
Stacking Ensemble (XGB + LGBM)	0,785	87,16	531,5

Table 1.13 – Performance du modèle Stacking Ensemble

1.5.10 Synthèse comparative et analyse critique

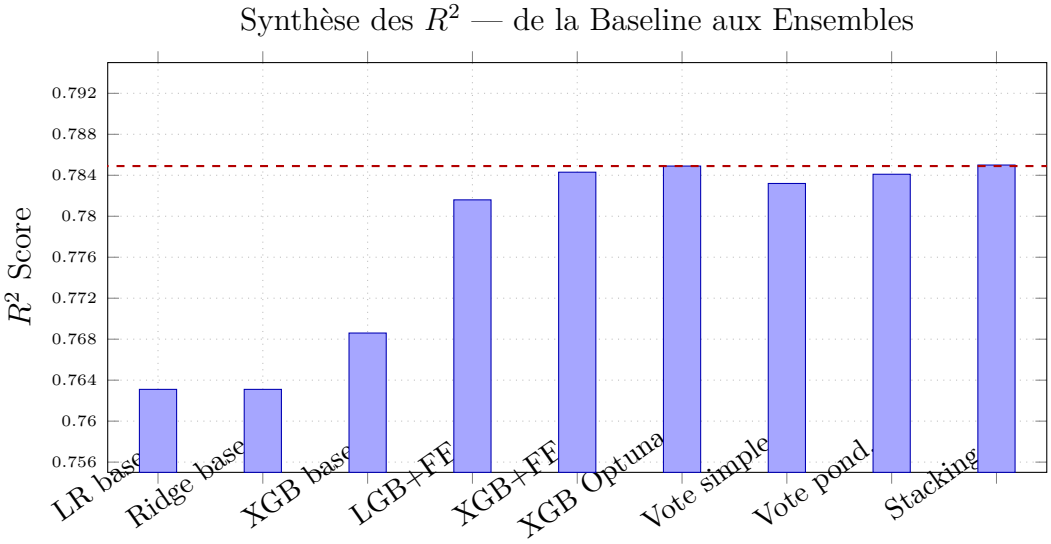

FIGURE 1.9 – Synthèse de tous les R^2 obtenus pour les différents modèles et ensembles

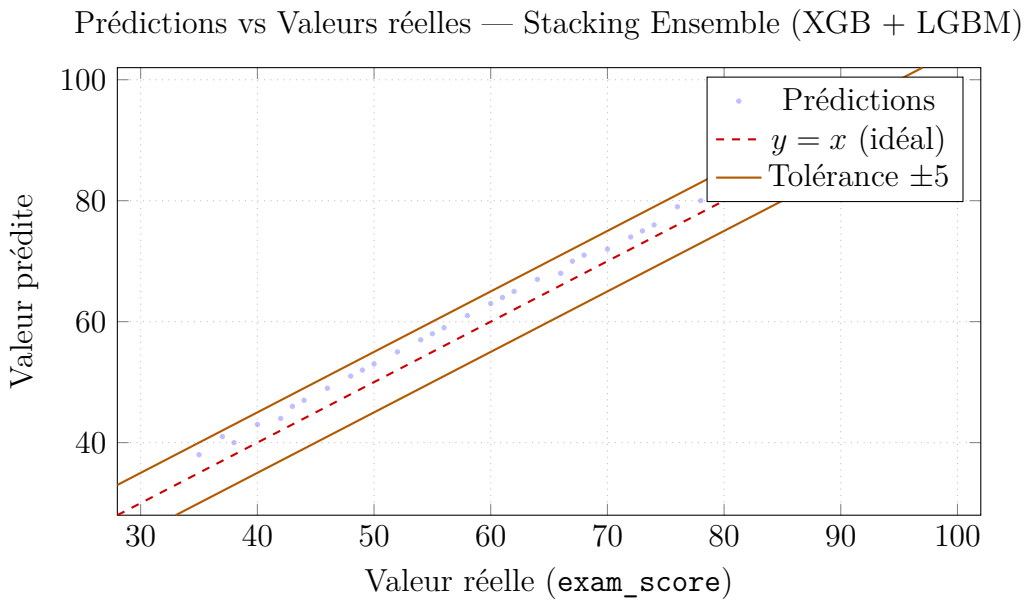

FIGURE 1.10 – Prédictions vs valeurs réelles — Stacking Ensemble (XGB + LGBM) (87,16 % des points dans la bande orange ±5)

Analyse critique : tous les modèles plafonnent autour de $R^2 \approx 0{,}785$. Ce plafond n'est pas algorithmique mais **intrinsèque au dataset**. Trois arguments l'étayent :

— Le R^2 de la Régression Linéaire (0,763) est proche du meilleur modèle (0,785) : les relations sont en grande partie linéaires. Les algorithmes complexes apportent un gain marginal.

— Les $\approx 21{,}5\,\%$ de variance inexpliquée correspondent probablement à des facteurs non mesurés dans le dataset (motivation intrinsèque, niveau initial, qualité pédagogique).

— L'Ensemble Learning n'apporte qu'un gain négligeable ($\Delta R^2 < 0{,}001$) car XGBoost et LightGBM exploitent les mêmes signaux : la diversification est insuffisante.

Les performances obtenues par l'approche de Stacking sur les jeux de données de test sont synthétisées dans le tableau ci-dessous, mettant en évidence la stabilité du modèle entre les phases publique et privée de la compétition.

Modèle	**Public Score**	**Private Score**
Stacking Regressor	8.78074	8.80970

Table 1.14 – Évaluation des performances finales sur la plateforme Kaggle

1.6 Conclusion

Ce chapitre a permis d'explorer, de prétraiter et de comparer plusieurs modèles pour la prédiction de `exam_score`. L'EDA a identifié `study_hours` et `class_attendance` comme variables les plus influentes. Le feature engineering a amélioré le R^2 de 0,763 à 0,785 grâce à cinq variables composites à formules explicites. **XGBoost optimisé par Optuna** est le modèle retenu : $R^2 = 0{,}7849$, MAE= 6,98, Acc.= 87,13 %.

Conclusion générale

Ce projet a permis de développer un pipeline complet de Machine Learning pour la prédiction de exam_score à partir des données de 630 000 étudiants.

Modèle final retenu : XGBoost optimisé (Optuna)

Métrique	**Valeur**
R^2 Score	0,7849
RMSE	8,77 pts
Accuracy ± 5	87,13 %

Table 1.15 – Récapitulatif des performances finales

Contributions méthodologiques :

- Cinq variables composites avec formules exactes et reproductibles.
- Encodage rigoureux : *OrdinalEncoder* pour les ordinales, *OneHotEncoder* pour les nominales, *StandardScaler* pour les numériques.
- Traitement des outliers par écrêtage IQR (conservation du volume).
- Optimisation bayésienne Optuna plus efficiente que GridSearch sur 630 000 lignes.

Limites et pistes d'amélioration :

1. **Plafond** $R^2 \approx 0{,}785$: l'intégration de données longitudinales (historique de notes, progression par semestre) pourrait lever ce plafond en capturant les facteurs non mesurés.
2. **Ensemble Learning peu diversifié** : ajouter un réseau de neurones dense ou un modèle à base de KNN pour une vraie diversification des estimateurs.
3. **Explicabilité** : appliquer SHAP (*SHapley Additive exPlanations*) pour quantifier la contribution de chaque feature aux prédictions individuelles — outil précieux pour les conseillers pédagogiques.

Ce travail constitue une base solide pour l'anticipation des performances académiques et fournit un outil d'aide à la décision concret pour l'accompagnement personnalisé des étudiants.